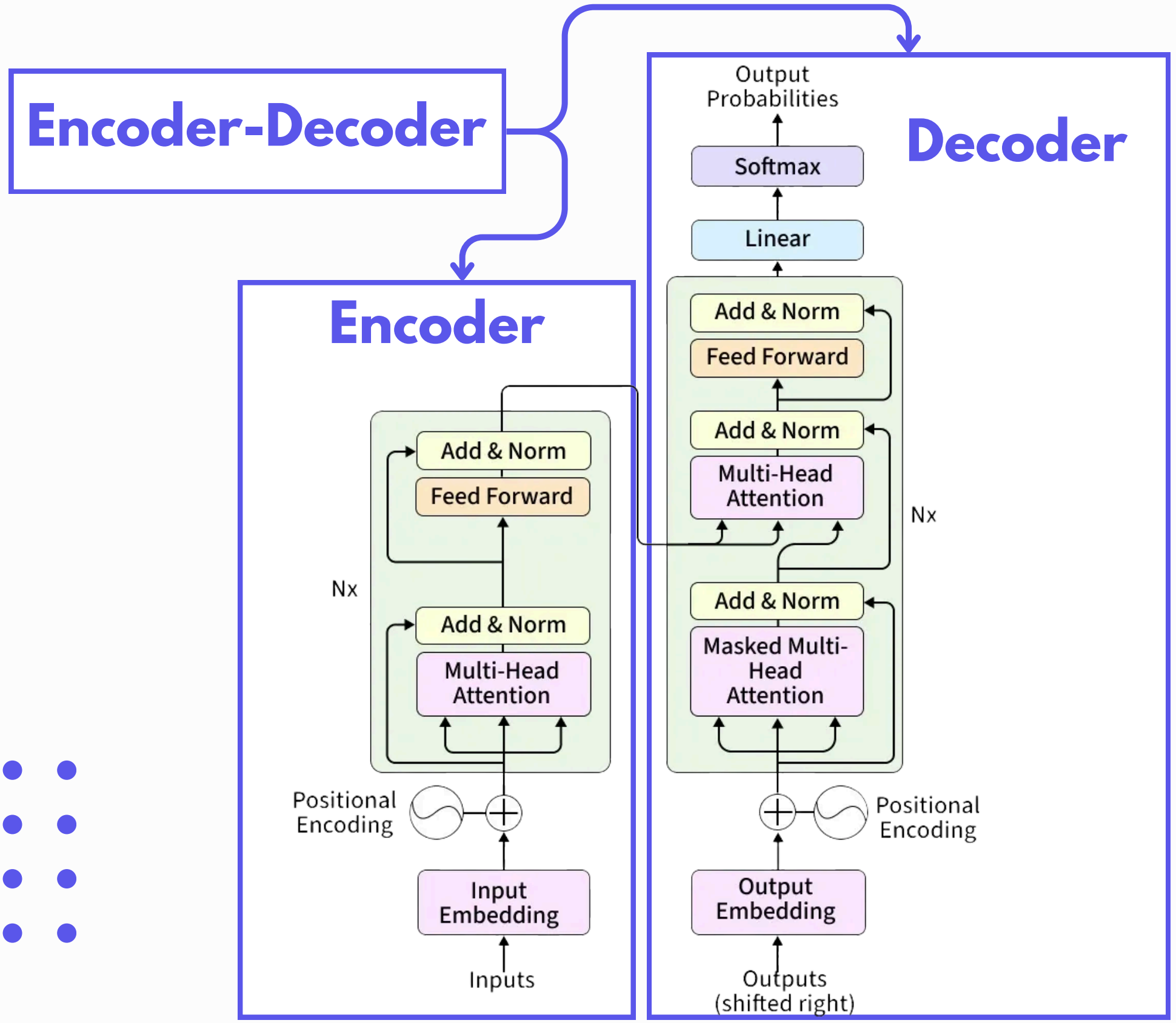
Understanding Transformers



The Architecture That Changed AI Forever

HRUTHIK GALI DILEEP FOLLOW

Repost

What is a Transformer?

A Transformer is a neural network architecture introduced in the 2017 paper "Attention is All You Need" by Vaswani et al.

It revolutionized NLP by replacing sequential processing with **parallel attention mechanisms**.

KEY INNOVATION

Instead of processing words one-by-one (like RNNs), Transformers look at **all words simultaneously** using self-attention.

Think of it like reading an entire paragraph at once and understanding how each word relates to every other word, rather than reading word by word.

REAL-WORLD ANALOGY

RNN: Reading a book page by page, remembering previous pages
Transformer: Scanning all pages simultaneously with perfect memory of relationships between any two sections

RNN vs LSTM vs Transformer

ARCHITECTURE	PROCESSING	MEMORY	SPEED
RNN	Sequential (one at a time)	Short-term, vanishing gradients	Slow
LSTM	Sequential with gates	Better long-term memory	Slow
Transformer	Parallel (all at once)	Perfect via attention	Fast

KEY PROBLEMS WITH RNN/LSTM

→ **Vanishing Gradients:** Struggles to learn long-range dependencies

→ **Sequential Processing:** Cannot parallelize, slow training

→ **Limited Context:** Forgets information from distant past

→ **Bottleneck:** All information compressed into fixed-size hidden state

HOW TRANSFORMERS SOLVE THIS

Self-Attention: Directly connects any two positions regardless of distance, enabling perfect long-range dependencies and parallel computation.

What Makes Transformers Special?

Four Revolutionary Features:

1. SELF-ATTENTION MECHANISM

Each word can attend to every other word in the sequence simultaneously.

Example: In "The animal didn't cross the street because it was too tired", the model learns "it" refers to "animal", not "street".

2. PARALLEL PROCESSING

Unlike RNNs that process sequentially, Transformers process all tokens at once, dramatically reducing training time.

3. POSITIONAL ENCODING

Since there's no sequential processing, position information is explicitly encoded so the model knows word order.

4. SCALABILITY

Can scale to billions of parameters and handle very long sequences efficiently with modern optimizations.

Self-Attention

THE ATTENTION FORMULA

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})\, V$$

COMPONENTS EXPLAINED

→ **Query (Q):** "What am I looking for?"

→ **Key (K):** "What information do I have?"

→ **Value (V):** "What is the actual information?"

REAL-WORLD ANALOGY

Scenario: You're in a library looking for books

Query: Your search criteria ("machine learning books")

Keys: Book titles on shelves

Values: Actual book contents

The model compares your query with all keys, finds the most relevant matches, and retrieves their values with appropriate weights.

Transformer Architecture Types

Three Main Architectures:

1. ENCODER-ONLY

Purpose: Understanding and classification tasks

Examples: BERT, RoBERTa, ALBERT

Use Cases: Text classification, named entity recognition, question answering

2. DECODER-ONLY

Purpose: Text generation and completion

Examples: GPT-3, GPT-4, LLaMA, Claude

Use Cases: Text generation, conversational AI, code completion

3. ENCODER-DECODER

Purpose: Sequence-to-sequence tasks

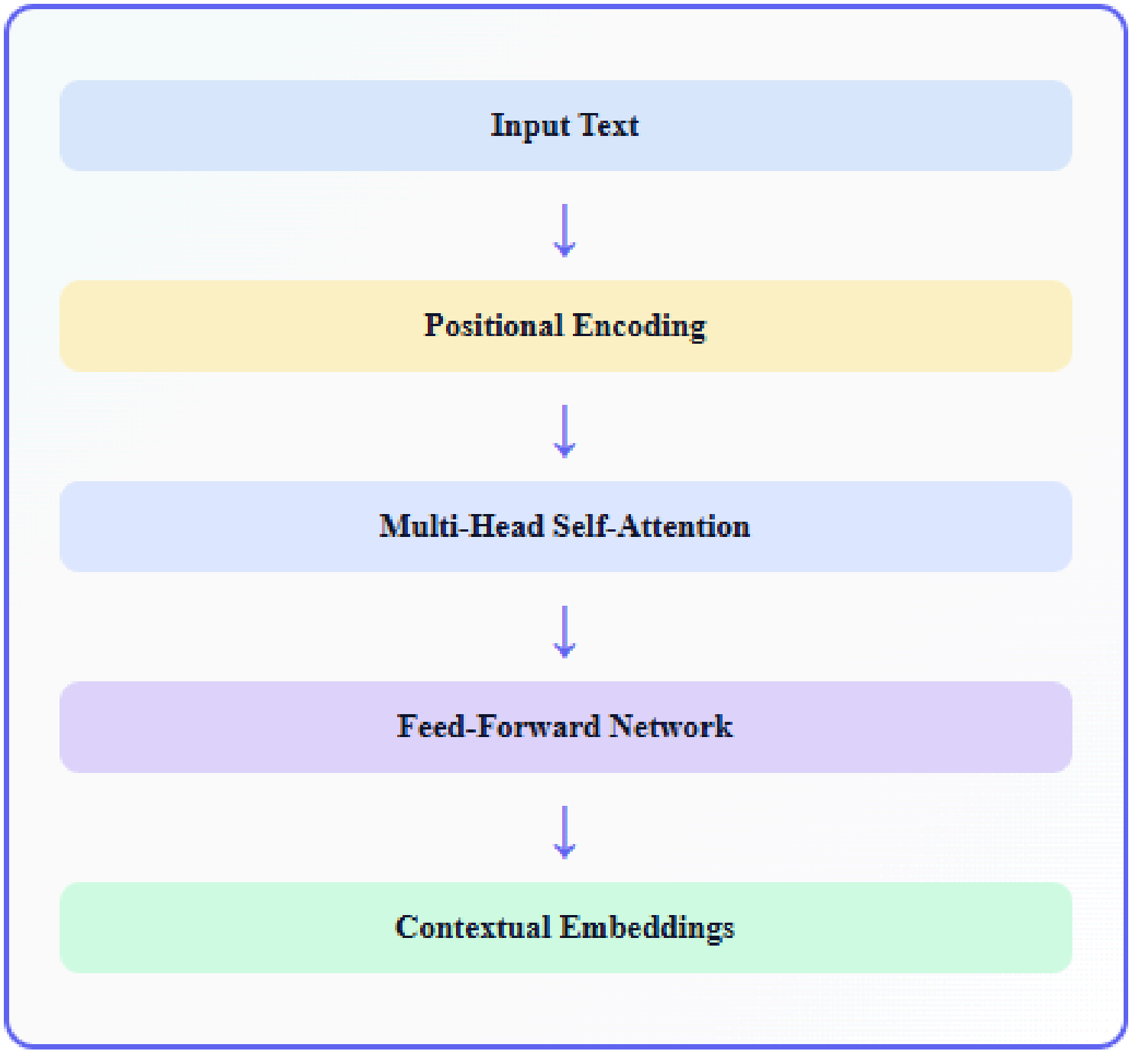
Examples: T5, BART, Original Transformer

Use Cases: Translation, summarization, text-to-text tasks

Encoder Architecture

KEY FEATURES

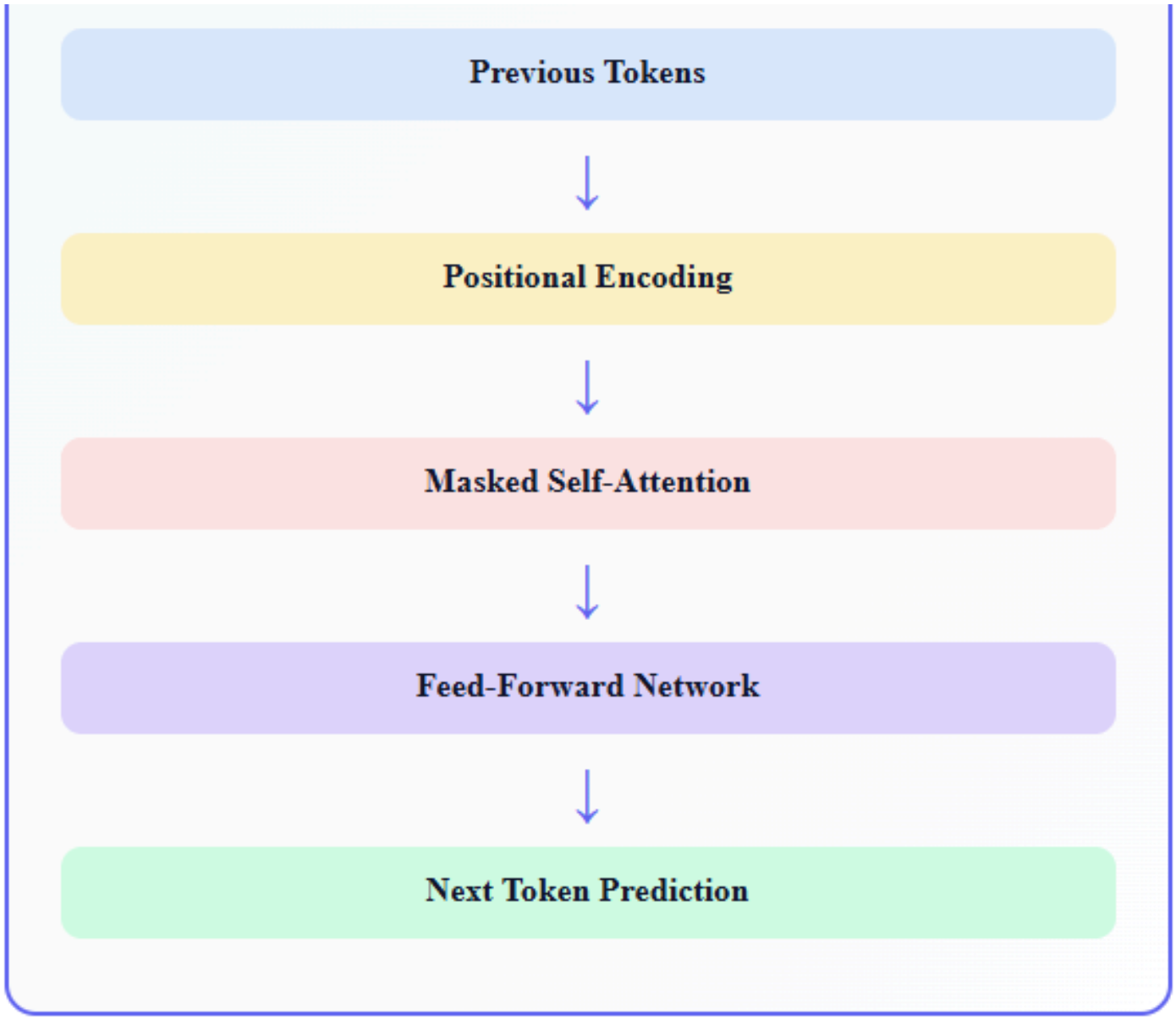
- **Bidirectional Attention:** Each token attends to all other tokens in the sequence
- **No Masking:** Complete visibility across the entire input
- **Contextual Understanding:** Creates rich representations for comprehension tasks
- **Parallel Processing:** All positions processed simultaneously



Decoder Architecture

KEY FEATURES

- → **Unidirectional:** Can only look at previous tokens (left-to-right)
- → **Masked Attention:** Prevents looking ahead during generation
- → **Autoregressive:** Generates one token at a time
- → **Causal:** Respects temporal order of sequence

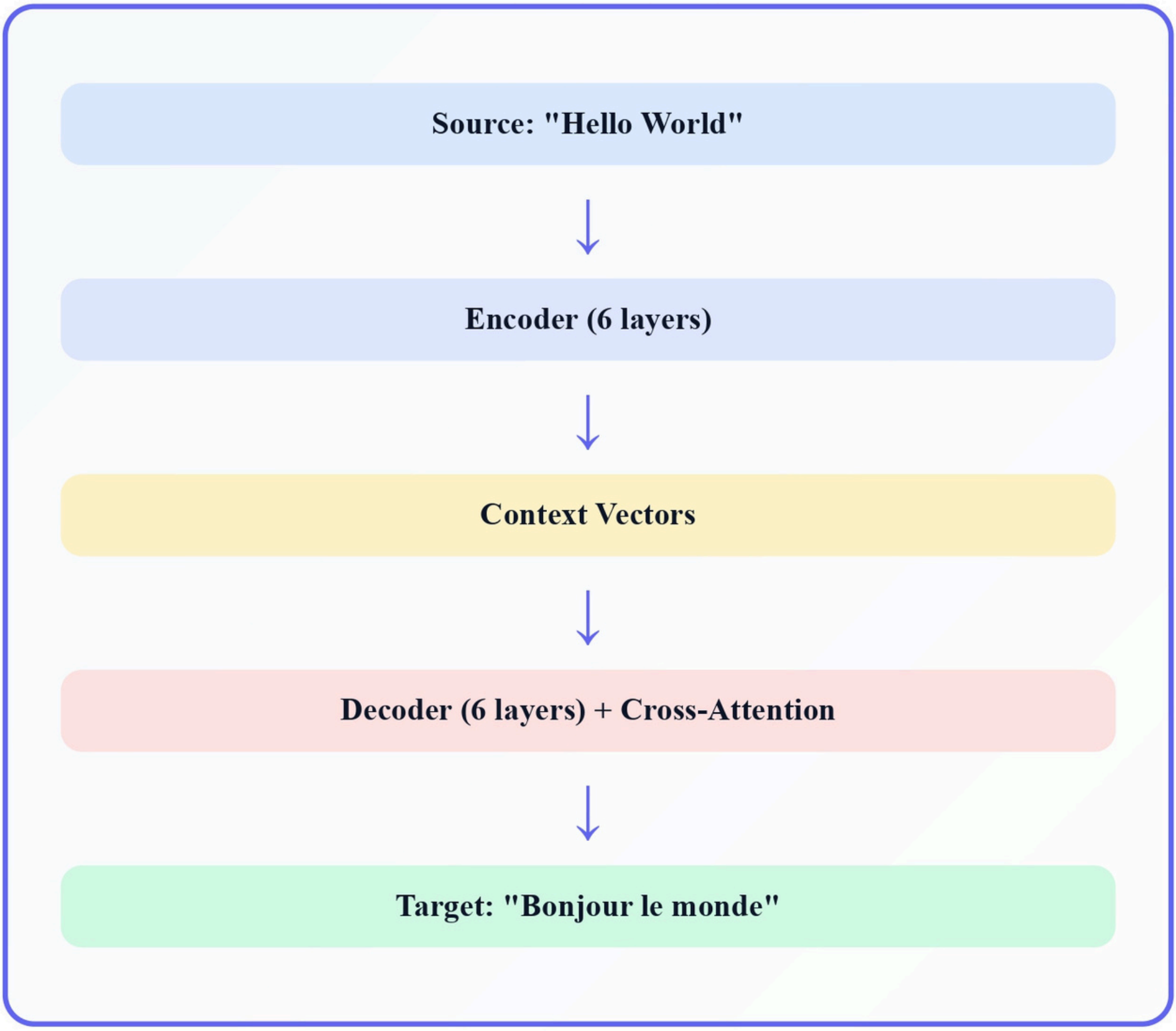


Encoder-Decoder Architecture

HOW IT WORKS

Encoder: Processes input sequence and creates contextual representations

Decoder: Generates output sequence while attending to encoder outputs



Causal Language Modeling

Causal Language Modeling (CLM) predicts the next word given all previous words, using only left-to-right context.

HOW IT WORKS

```
Input: "The cat sat on the"
Predict: "mat" (or "floor", "chair", etc.)
```

The model can only see words before the current position. It's like reading a book where you can only look at previous pages.

USED BY

GPT series, LLaMA, Claude, PaLM - all autoregressive models

REAL-WORLD ANALOGY

Like writing an email: you can see everything you've written so far, but not what you'll write next. Each word builds on previous context naturally.

Masked Language Modeling

Masked Language Modeling (MLM) **randomly masks words and predicts them using bidirectional context (left and right).**

HOW IT WORKS

```
Original: "The cat sat on the mat"
Masked: "The cat [MASK] on the mat"
Predict: "sat"
```

The model sees context from both directions, making it better at understanding but not suitable for generation.

USED BY

BERT, RoBERTa, ALBERT - all bidirectional encoder models

REAL-WORLD ANALOGY

Like fill-in-the-blank questions: "The capital of France is ___". You use context from both before and after the blank to determine the answer.

Why 6 Encoder/Decoder Layers?

THE ORIGINAL CHOICE

The original 2017 Transformer paper used **6 encoder layers** and **6 decoder layers** based on empirical experiments.

This was a balance between model capacity and computational efficiency with 2017 hardware.

WHAT EACH LAYER DOES

→ **Early layers:** Capture low-level patterns (syntax, grammar)

→ **Middle layers:** Learn semantic relationships and dependencies

→ **Deep layers:** Form abstract, task-specific representations

MODERN VARIATIONS

→ BERT-Base: 12 encoder layers

→ BERT-Large: 24 encoder layers

→ GPT-3: 96 decoder layers

→ Claude & GPT-4: Likely 80-100+ layers

Scaling Transformers

WHY SCALE?

More layers = better performance, but with diminishing returns. The key is finding the optimal balance for your task and resources.

MODEL SIZE	LAYERS	PARAMETERS	CAPABILITY
Small	6-12	100M-1B	Basic tasks
Medium	12-24	1B-10B	Most tasks
Large	24-48	10B-100B	Complex reasoning
Very Large	48-100+	100B-1T+	SOTA performance

SCALING LAWS

Performance improves predictably with model size, data, and compute. Each 10x increase in size yields consistent improvements in capability.

Autoregressive Models

Autoregressive means the model generates output sequentially, where each new token depends on all previously generated tokens.

GENERATION PROCESS

```
Step 1: "The" → predict next → "cat"
Step 2: "The cat" → predict next → "sat"
Step 3: "The cat sat" → predict next → "on"
Step 4: "The cat sat on" → predict next → "the"
Step 5: "The cat sat on the" → predict next → "mat"
```

KEY CHARACTERISTICS

- → Each token depends on all previous tokens
- → Cannot parallelize generation (sequential by nature)
- → Flexible for open-ended generation
- → Used by GPT, LLaMA, Claude, and similar models

Practical Applications

ENCODER-ONLY (BERT-STYLE)

- → Text classification (sentiment analysis, spam detection)
- → Named entity recognition
- → Question answering
- → Semantic search

DECODER-ONLY (GPT-STYLE)

- → Text generation and creative writing
- → Conversational AI and chatbots
- → Code generation and completion
- → Summarization and rewriting

ENCODER-DECODER (T5-STYLE)

- → Machine translation
- → Text summarization
- → Document generation from structured data
- → Text-to-text transformation tasks

CROSS-DOMAIN APPLICATIONS

Vision Transformers (ViT), Audio Transformers, Multimodal models (CLIP, GPT-4V), Protein folding (AlphaFold2)